

PROJECT COMPONENTS REFERENCE

Every layer, every file, every technique

What You Built

End-to-end AI P&L; copilot — full component map

A complete reference of every file, every layer, and every non-obvious engineering decision in the ABC Fitness Club Analytics project. Use it as a study aid, interview prep, resume input, or hand it to a teammate who wants to onboard.

AT A GLANCE

7 layers · 30+ Python modules · 5 Streamlit pages · ~\$10/mo to run

Python · Streamlit · Claude Haiku 4.5 · Altair · ReportLab · openpyxl

Table of contents

1.	Architecture at a glance — the 7 layers	3
2.	Layer 1 · Data Pipeline (raw → SLV → Gold)	4
3.	Layer 2 · Data Reader	5
4.	Layer 3 · Analytics	6
5.	Layer 4 · LLM (Claude Haiku 4.5)	7
6.	Layer 5 · Dashboard UI (Streamlit)	9
7.	Layer 6 · Export (PDF, logo)	11
8.	Layer 7 · Orchestration	12
9.	Deployment & hosting patterns	13
10.	Full tech stack	14
11.	Non-obvious techniques (the interview gold)	15
12.	What this project proves you can do	17
13.	One-liner pitches you can copy-paste	18

1. Architecture at a glance — the 7 layers

The project is organized into seven clean layers. Each lower layer is a strict dependency of the layer above — nothing reaches sideways. This separation is why you can swap out (for example) Excel for Postgres without touching the dashboard, or swap Claude for a different LLM without touching the data readers.

#	Layer	Responsibility	Key file
7	UI	Streamlit multi-page app + PDF export	Dashboard.py
6	Export	Markdown → branded PDF, logo generation	pdf_export.py
5	LLM	Anthropic SDK wrapper + prompts + jobs	llm_client.py · prompts.py
4	Analytics	Flagging, anomalies, partial-month rules	analyzer.py · month_context.py
3	Reader	Strongly-typed Excel access	pnl_reader.py · membership_reader.py
2	Pipeline	Weekly raw .xlsx → Gold master	pipeline.py
1	Storage	Local Excel files (medallion folders)	Raw / SLV / Gold

Why this matters

This is the same layered design Databricks/Snowflake/Microsoft Fabric use internally. You've already built a medallion architecture — just on disk instead of in the cloud. Migrating later is mostly a swap of layers 1 and 2; everything above stays.

2. Layer 1 · Data Pipeline (raw → SLV → Gold)

This is the ETL — turning the chaotic weekly raw files (downloaded from the accounting system) into a clean, queryable monthly master Excel.

File	What it does	Tech used
pipeline.py	Reads weekly raw .xlsx files via Excel COM (preserves cached formula values), deduplicates rows by content fingerprint, builds Ultra + Premium aggregated tabs, maintains weekly + daily reconciliation tabs, writes the monthly master.	Python, win32com, openpyxl
pipeline_dummy.py	Same pipeline but for the 52-week synthetic prototype dataset (used for development without touching real data).	Python, openpyxl
Folder structure	1. Ingestion(Raw) → 2. Transformation(SLV) → 3. Golden Data(GLD). Medallion-style layering on disk.	Filesystem convention

What this proves

You can design and implement a multi-stage ETL pipeline with clear separation between raw, cleaned, and business-ready layers — the same pattern enterprise lakehouses use. You've also solved real Windows-Excel interop problems (COM filter clearing, formula-value preservation, content fingerprinting for dedup) that most data engineers never touch.

3. Layer 2 · Data Reader

The readers are the only code that knows the structure of the master Excel. Everything above this layer works with typed Python dataclasses. If the Excel schema changes, only this layer needs an update.

File	What it does	Tech
pnl_reader.py	Reads the P&L; tab. Returns a list of MonthlyPnL dataclasses (month, ultra_rev, premium_rev, 8 expense lines, derived total_exp / profit / gm_pct as @property).	openpyxl, dataclasses
membership_reader.py	Reads Ultra + Premium tabs. Builds per-client aggregates (ClientAggregate) and SegmentBreakdown (new vs existing × month).	openpyxl, dataclasses
qa_context.py	RAG retriever — assembles the full context block (P&L; summary + member aggregates + all generated briefs) for the Q&A; chat.	Pure Python
data_sources.py	Single source of truth for paths. Resolves the active source via session_state > env var > default. Defines a DataSource dataclass.	dataclasses + lazy resolution
output_paths.py	Uses PEP 562 module-level __getattr__ to re-resolve BRIEFS_DIR / ANOMALIES_DIR / ANNUAL_DIR on every access — so switching the source dropdown actually changes the directory without a restart.	Module magic (PEP 562)

Design pattern

Single-source-of-truth + lazy resolution. Every page just does from output_paths import BRIEFS_DIR — the value is fresh on every page run. No global mutable state, no manual cache invalidation.

4. Layer 3 · Analytics

Pure Python — no LLM, no Spark, no fancy libraries. Just deterministic math that the LLM can't get wrong because we don't ask it to. This separation is the line between 'production AI' and 'demo AI'.

File	What it does	Notable rules
analyzer.py	Month-over-month + rolling-3M comparison for every line. Classifies each as up_significant / down_significant / steady / new / vanished.	≥10% MoM threshold
year_analyzer.py	FY totals, quarterly breakdown, outlier detection, loss months, best/worst month per metric.	Statistics module
anomaly_detector.py	6 rule-based detectors: z-score outlier, GM collapse, new-line spike, unexpected zero, negative profit, revenue collapse. Each rule has a severity (critical/high/medium/low).	z-score, multiplier rules
month_context.py	The CONSULTING BRAIN. Computes is_partial_month + unposted_lines + pace_factor. Detects which expense lines are zero now AND were non-zero in historical months — labels them as 'unposted' so the LLM doesn't claim 'salary dropped 100%'.	Calendar math + history compare

The month_context.py innovation

This file is the difference between a script that 'calls an API' and a system that 'thinks like a consultant'. It detects partial-month state deterministically, identifies which expense lines are unposted, and feeds these facts to the LLM as MONTH STATUS — so the model frames insights correctly instead of computing nonsense MoM% comparisons on incomplete data.

5. Layer 4 · LLM (Claude Haiku 4.5)

All Anthropic API integration lives here. Cost-optimized via prompt caching; structured-output JSON schemas keep responses parseable; the system prompts contain the business rules + banned-phrase lists.

File	What it does
prompts.py	All system prompts (monthly, owner-annual, accountant-annual, anomaly, Q&A;). Includes partial-month banned-phrase rules and two output-format variants. Both English and Hindi instructions baked in.
llm_client.py	Thin wrapper around anthropic.messages.create. Sets cache_control: ephemeral on the system prompt (~80% cost reduction on repeat Q&A;).
narrate_month.py	CLI — generates the monthly brief. Calls month_context.compute_context, passes is_partial to build_user_prompt (which masks MoM% data in partial mode), then calls Claude.
sentinel.py	CLI — anomaly sentinel. Runs anomaly_detector first (rule-based), then calls Claude for triage with the partial_month_artifact classification.
narrate_year.py	CLI — annual review. Generates TWO versions of the same data: Owner version (narrative) and Accountant version (numerical).
ask_pnl.py	Terminal REPL — interactive Q&A; with prompt caching across turns. Output via JSON schema ({answer, next_questions: [...]}).

Three layers of LLM safety

When the partial-month bug was fixed, the solution worked at THREE levels to ensure the LLM couldn't hallucinate a comparison:

- **Rule layer (system prompt)** — explicit BUSINESS RULES section with a hard-banned-phrase list ("decline", "drop", "below prior", etc.).
- **Template layer (system prompt)** — "What moved" section has TWO variants. Partial month → "Where things stand (to date)" with no emoji, no MoM%.
- **Data layer (user prompt)** — `_line_row(partial=True)` strips `prior_month` and `pct_change` columns. The model cannot cite a decline because we don't give it one.

Cost engineering

Total operating cost for one fitness centre: ~\$5-10/month. Achieved via Haiku 4.5 (cheapest capable model), prompt caching, and structured outputs.

6. Layer 5 · Dashboard UI (Streamlit)

Six Streamlit pages, one shared theme. Soft-light palette, sky-blue accent, Indian-number formatting, mobile-aware CSS, bilingual UI.

Page	What you see
Home (Dashboard.py)	Hero KPI strip (Total Revenue · Operating Cost · Net Profit · Gross Margin), each with dual-format display (■13.30L headline + ■13,29,810 actual + delta% vs prior). Below: revenue+profit dual line chart with data labels, GM% bars, segment mix donut + stacked area.
1. Monthly Briefs	Month selector dropdown → KPI tiles for selected month → mini 6-month revenue trend chart → brief markdown content → PDF download.
2. Annual Reviews	FY dropdown × audience picker (Owner / Accountant / Side-by-side). FY headline KPIs + revenue+profit chart. PDF download per audience.
3. Anomaly Reports	Severity-graded KPI strip + filterable list of monthly reports with PDF export.
4. Q&A; Chat	RAG-grounded chat. Suggested follow-up questions from the LLM. Token usage tracker. PDF export of full conversation.
5. Insights & Summary	12-section interactive analytics: Executive Summary KPIs, Profit Bridge waterfall, Cost Pareto (80/20), Trend Focus, Revenue Composition, Cost Structure, Profitability, MoM Bridge, Run-rate, Quarterly view, Cost Efficiency, Scenario Sliders, Key Insights callouts.

theme.py — the design system

One file holds the entire visual identity. Key elements:

- **Soft-light palette** — off-white background (#F8FAFC), sky-blue accent (#0EA5E9), slate text. Semantic colors for success/warn/danger.
- **kpi_tile()** component — dual format: short value (■13.30L) + full actual (■13,29,810) + optional delta. Used 40+ times across pages.
- **Indian number formatting** — `fmt_short_inr`, `fmt_inr` (lakh-style commas), `fmt_chart_label` for compact labels.
- **i18n via t() function** — `T = {"en": {...}, "hi": {...}}`. Sidebar toggle switches language without page reload.
- **Mobile-responsive CSS** — `@media` (max-width: 768px) shrinks hero title, KPI values, chart label fonts. Columns stack naturally.
- **Source picker** — sidebar dropdown to switch between Current Year and Prior Year. Triggers cache clear + rerun.

Chart engineering

Every chart has data labels rendered directly on the visual — bars show values above/below, donuts show percentages inside wedges, lines show values above each point. No 'hover to see' — the story is readable instantly.

7. Layer 6 · Export

File	What it does	Tech
pdf_export.py	TWO entry points: <code>markdown_to_pdf(content, title, subtitle)</code> renders any .md as a brand-aligned PDF. <code>chat_to_pdf(messages)</code> renders a conversation transcript with user/AI bubbles. Same color palette.	ReportLab + markdown lib
generate_logo.py	One-time script — generates the <code>assets/logo.png</code> . PIL draws a black circle with split yellow/red border, a bicep silhouette, the brand letters in yellow Impact font with red drop shadow, plus a white 'FITNESS · CLUB' tagline and a yellow italic SINCE 2020.	PIL/Pillow

Why ReportLab? Pure-Python, no LaTeX, no headless Chrome, no wkhtmltopdf binary. Works the same on Windows / Linux / Streamlit Cloud / a Docker container — zero deployment surprises.

8. Layer 7 · Orchestration

File	What it does
refresh_all.py	The conductor. Three modes via CLI flags: pipeline+AI (full), pipeline only (--skip-ai), or AI only (--skip-pipeline). Internally it's a generator function yielding (step_label, success, detail) — so the dashboard's Refresh button can drive a live progress panel.
Sidebar Refresh button	Calls refresh_all.run_refresh() generator. Streams progress to a Streamlit container, then clears the cache and reruns the page.

Production thinking

Even without a real scheduler, the orchestrator already obeys the rules real production DAGs need: idempotent, restartable, observable, CLI-or-library (works as a script for cron AND as an import for the UI).

9. Deployment & hosting patterns

Pattern	Why it exists
Three independent workspaces	Production, Dummy (52-week prototype), Original combined. Each is fully self-contained — deleting any one doesn't break the others.
Versioned snapshots (v1...v6)	Each major design milestone saved to versions/ with a rollback recipe in VERSIONS.md. Pure-file equivalent of git tags.
unified workspace	Final consolidated dashboard combining current + prior year via a sidebar dropdown. Single URL, instant switch via per-source AI-Output subfolders.
Cloudflare Tunnel (recommended)	Free permanent HTTPS URL pointing at the local PC. No port forwarding, no public hosting, data stays local.
Streamlit Community Cloud (alt)	Free public hosting via GitHub auto-deploy. Used for the portfolio demo.
GitHub portfolio (private)	Sanitized version: 517 member names anonymized, 98 locations bucketed into Branch A-H, brand renamed to ABC Fitness Club.
Future: Azure Databricks + UC + ADLS Gen2	21-page learning PDF maps out the multi-tenant SaaS path.

10. Full tech stack

Layer	Tools / Libraries	Why this choice
Language	Python 3.12	Ecosystem for data + LLM. Modern type hints.
UI	Streamlit 1.30+	Single-file pages, fast iteration.
LLM	Claude Haiku 4.5 (Anthropic SDK)	Cheapest capable model, 200K context, prompt caching, structured outputs.
Data	openpyxl, pandas, dataclasses	openpyxl for full Excel control; pandas for chart data; dataclasses for type-safe domain objects.
Charts	Altair (Vega-Lite)	Declarative, theme-aware, data labels on every encoding.
PDF	ReportLab + markdown	Pure Python. No LaTeX, no Chrome binaries.
Images	PIL/Pillow	Logo generation.
Excel COM	pywin32	Windows-only — used in pipeline.py to preserve cached formula values.
Config	python-dotenv	.env file. Loaded with override=True.
Hosting	Cloudflare Tunnel · Streamlit Cloud · GitHub	Free tier across the board. Total ~\$10/mo to operate.
Browser auto	Playwright	Auto-grab the 6 README screenshots.
Source control	Git + GitHub	Portfolio repo + on-disk versions/.

11. Non-obvious techniques — the interview gold

These are the parts that show real engineering judgment, not just gluing tutorials together. Each one is worth its own talking point.

#	Technique	What makes it interesting
1	PEP 562 module-level <code>__getattr__</code>	<code>output_paths.py</code> uses Python's PEP 562 to make <code>BRIEFS_DIR</code> dynamic. Streamlit re-runs page scripts on every interaction; the <code>from-import</code> triggers <code>__getattr__</code> each time, returning the fresh path for whichever data source is currently active. No restart needed when the dropdown changes.
2	Three layers of LLM defense (partial months)	Banned-phrase list in the system prompt + template variants + data masking at the user-prompt level. The model literally cannot cite a decline because the prior-month column is missing from the data we feed it.
3	Prompt caching for ~80% Q&A; cost reduction	Every system prompt sent with <code>cache_control: ephemeral</code> . First call seeds cache (full input billed); follow-up calls pay ~10% of normal input cost.
4	Structured outputs via JSON schema	The Q&A; chat uses <code>output_config.format.type = json_schema</code> . No string parsing, no regex, no 'sometimes the LLM forgets the format'.
5	Deterministic business-rules layer (<code>month_context.py</code>)	Detects partial months + unposted lines + computes pace factor BEFORE the LLM is involved. Then injects these facts as context.
6	Indian-number dual format	Every KPI tile shows ■13.30L as headline + ■13,29,810 as actual. Owner gets the gut-feel number AND the audit number in one glance.
7	Single <code>t()</code> function for <code>i18n</code>	<code>T = {"en": {...}, "hi": {...}}</code> . Sidebar toggle updates <code>session_state</code> + reruns. All UI strings flow through one code path.
8	Sanitized portfolio version	517 real member names → 'Member 0001' via deterministic map. 98 locations → 'Branch A-H'. Single one-off script, reproducible.
9	PDF rendering of both markdown reports AND chat transcripts	Single ReportLab pipeline. Same brand colors, same logo header, same accent rule. One implementation, two use cases.
10	Mobile-responsive Streamlit via <code>@media</code> query	Standard CSS <code>@media (max-width: 768px)</code> inside <code>inject_theme()</code> shrinks <code>h1</code> /KPI values/chart label fonts on phones.

12. What this project proves you can do

For your resume, LinkedIn, or interview prep — each row maps a skill cluster to specific evidence in this codebase.

Skill cluster	Evidence in this project
Data engineering	Multi-stage pipeline (Raw → SLV → Gold). Reconciliation tabs. Excel COM automation. Content-fingerprint deduplication.
AI / LLM engineering	Prompt engineering (banned-phrase rules). RAG. Structured-output JSON schemas. Prompt caching. Multi-prompt design (3 audiences from 1 dataset). Business-rules layer.
Full-stack development	Streamlit 6-page app. Custom theme. Mobile-responsive CSS. PDF export. i18n bilingual UI.
Product thinking	Indian-number formatting. Dual KPI display. Partial-month intelligence. 'Consulting voice' outputs. Empty-state messages framed for the user.
Software architecture	7 clean layers, no sideways dependencies. Single source of truth. Dynamic module attributes (PEP 562). Source-switching without restart.
DevOps / deployment	Three isolated workspaces. Versioned snapshots. Cloudflare Tunnel design. Streamlit Cloud deployment. Multi-environment isolation.
Documentation	README with badges + screenshots. ARCHITECTURE.md deep-dive. Learning PDFs. Code comments at the right level.
Cost engineering	■10/month operating cost via Haiku 4.5 + prompt caching + free tiers.
Privacy / security	Local-first data. Secrets in .env (gitignored). Sanitized portfolio with anonymized PII.
Browser automation	Playwright-based screenshot capture for the README. Element-based scroll, retry on async rendering.

13. One-liner pitches (copy-paste ready)

For a resume bullet (~30 words)

Resume version

Built end-to-end AI P&L; copilot for small-business fitness centres — Python ETL pipeline → Claude Haiku 4.5 with deterministic business-rules layer → 6-page Streamlit dashboard with PDF export. Operating cost ~₹10/month. ~30 modules, MIT licensed.

For LinkedIn About / Featured Project (~60 words)

LinkedIn version

Shipped an MBB-grade P&L; copilot for small-business fitness centres. Excel ingestion pipeline → 6-layer Python data platform → Claude Haiku 4.5 with prompt caching + structured outputs + a deterministic business-rules layer that teaches the LLM when NOT to compare (partial months, unposted month-end lines). Streamlit dashboard with 12-section interactive analytics, EN/BI bilingual, branded PDF export. ~₹10/month.

For an interview opener (~15 seconds spoken)

Spoken version

I built an AI-powered P&L; copilot that turns a small fitness centre's weekly Excel files into consultant-style monthly briefs, anomaly reports, and a dashboard — all powered by Claude Haiku 4.5 — for about ten rupees a month. The interesting engineering challenge was teaching the LLM when NOT to compare, when a month is only partially complete.

For X / Twitter (~280 chars)

X version

Built an AI P&L; copilot for SMB fitness centres. Python + Streamlit + Claude Haiku 4.5. The hardest part: teaching the LLM NOT to compare partial months to complete ones. 3-layer defense. Runs for ~₹10/month.

End of components reference. Generated 26 May 2026 · Self-study, interview prep, teaching aid. MIT licence.